

Name:Madhavi katekhaye

Roll no:22

Section:b

Batch:b2

ML Lab 3

Aim:

To implement Find-S and Candidate Elimination Algorithms on the Enjoy Sports dataset and compare the generated hypotheses

2. Objectives

The objectives of this experiment are:

- 1.To understand the concept of **concept learning** in machine learning.
- 2.To implement the **Find-S algorithm** using the EnjoySport dataset.
- 3.To implement the **Candidate Elimination algorithm** using the same dataset.
- 4.To identify the **most specific hypothesis** using the Find-S algorithm.
- 5.To determine the **specific and general boundaries** using Candidate Elimination.
- 6.To observe how positive and negative training examples affect the hypotheses.
- 7.To compare the hypotheses generated by both algorithms.
- 8.To understand the role of **specific (S) and general (G) boundaries** in concept learning.

9.Dataset / Resources Used

10.Dataset Used: EnjoySport Dataset

11. The **EnjoySport dataset** is used to demonstrate concept learning algorithms. The dataset contains **10 training examples and 7 columns**. Six columns represent input attributes, while **EnjoySport** is the target attribute. The uploaded file confirms that all seven columns contain non-null categorical/object data.

Attribute	Description
Sky	Weather condition such as Sunny, Rainy, Cloudy
AirTemp	Air temperature such as Warm or Cold

Attribute	Description
Humidity	Humidity condition such as Normal or High
Wind	Wind condition such as Strong or Weak
Water	Water temperature such as Warm or Cool
Forecast	Weather forecast such as Same or Change
EnjoySport	Target/output: Yes or No

The dataset used in the uploaded file contains examples such as **Sunny, Warm, Normal, Strong, Warm, Same → Yes** and **Rainy, Cold, High, Strong, Warm, Change → No**.

Software/Resources

- **Python**
- **Jupyter Notebook / Google Colab**
- **Pandas** - used to read and examine the CSV dataset.
- **NumPy** - used in implementing the algorithms.
- **EnjoySport CSV dataset** - used as the training dataset.

In the uploaded implementation, the dataset is loaded using `pd.read_csv("enjoy.csv")`, and Pandas functions such as `head()`, `tail()`, `info()`, and `describe()` are used to examine the data.

Final Result from Your File

The **Find-S algorithm** produces the final hypothesis:

[Sunny, Warm, ?, ?, ?, ?]

The Candidate Elimination implementation produces a **specific boundary (S)** beginning with:

[Sunny, Warm, Normal, Strong, Warm, Same]

and maintains a **general boundary (G)** beginning with:

[?, ?, ?, ?, ?, ?]

Date of performance:

6/8/2026

Github link

1.

```

importin
g Pandas

```

```
In [1]: import pandas as pd
df = pd.read_csv("enjoy.csv")
print(df.head())
```

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes
4	Sunny	Cold	Normal	Weak	Warm	Same	

2. Reading the Dataset

```
In [4]: df.head()    # First 5 rows
df.head(10)

df.tail()
df.tail(10)
```

Out [4]: **Sky AirTemp Humidity Wind Water Forecast EnjoySport**

0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes
4	Sunny	Cold	Normal	Weak	Warm	Same	No
5	Cloudy	Warm	Normal	Weak	Warm	Same	No
6	Sunny	Warm	Normal	Weak	Cool	Same	Yes

```
df.info()
df.describe()
df.describe(include='all')
```

7	Rainy	Warm	High	Weak	Cool	Change	No
8	Sunny	Warm	Normal	Strong	Cool	Same	Yes
9	Sunny	Warm	High	Weak	Warm	Same	Yes

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10 entries, 0 to 9 Data columns (total 7
columns): In [5]:
```

#	Column	Non-Null Count	Dtype
0	Sky	10 non-null	object
1	AirTemp	10 non-null	object
2	Humidity	10 non-null	object
3	Wind	10 non-null	object
4	Water	10 non-null	object
5	Forecast	10 non-null	object
6	EnjoySport	10 non-null	objectdtypes: object(7)

memory usage: 688.0+ bytes

Out[5]:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
--	-----	---------	----------	------	-------	----------	------------

count	10	10	10	10	10	10	10
unique	3	2	2	2	2	2	2
top	Sunny	Warm	Normal	Strong	Warm	Same	Yes
freq	7	8	5	5	6	7	6

```
In      X=df.iloc[:, :-1].values
[9]:    y=df.iloc[:, :-1].values
```

Find-S Algorithm

11. Selecting Input Features

2. Selecting Target Values

```
y = df.iloc[:, -1].values
```

Description:

This statement selects the **last column** of the dataset and stores it in y. The last column is **EnjoySport**, which contains the target values **Yes/No**.

Your file also contains an earlier line `y=df.iloc[:, :-1].values`, but it is subsequently overwritten by the correct target-selection statement above.

. Defining the Find-S Function

Find-S Algorithm - Step-by-Step Code Description

1.Import NumPy

NumPy is imported for handling array-based data.

2.Define `find_s()` function

A function is created with X and y as inputs.

3.Initialize hypothesis

The hypothesis is initially set to None.

4. Loop through training examples

A loop processes each example in the dataset one by one.

5. Check positive examples

The program checks whether the target value is "Yes".

6. Set initial hypothesis

For the first positive example, its attribute values are copied as the initial hypothesis.

7. Compare attributes

Each attribute of the hypothesis is compared with the current positive example.

8. Generalize different attributes

If two attribute values are different, the hypothesis value is changed to ?.

9. Display hypothesis

The current hypothesis is printed after processing each training example.

10. Return final hypothesis

After processing all examples, the function returns the final hypothesis.

11. Execute the function

`find_s(X, y)` is called and its result is stored in `final_hypothesis`.

12. Display final result

The final hypothesis is printed. In your output, it is **[Sunny, Warm, ?, ?, ?, ?]**.

In []:
]:

In []:

```
In [14]: X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

import numpy
as np def
find_s(X, y):
    hypothesis = None
    print("Initial Hypothesis :", hypothesis)

    for i in range(len(X)):
        if y[i].lower() == "yes":    # consider only
            positive examples if hypothesis is None:
                hypothesis = X[i].copy()
            else:
                for j in range(len(hypothesis)):
                    if hypothesis[j] != X[i][j]:
                        hypothesis[j] = "?"
        print(f"\n After Training Ex. {i+1}")
        print(hypothesis)
    return hypothesis
final_hypothesis =
find_s(X, y)
print("\n Final Hypothesis")
print(final_hypothesis)
```

Initial Hypothesis : None

After Training Ex. 1
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

After Training Ex. 2
['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']

After Training Ex. 3
['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']

After Training Ex. 4
['Sunny' 'Warm' '?' 'Strong' '?' '?']

After Training Ex. 5
['Sunny' 'Warm' '?' 'Strong' '?' '?']

After Training Ex. 6
['Sunny' 'Warm' '?' 'Strong' '?' '?']

After Training Ex. 7
['Sunny' 'Warm' '?' '?' '?' '?']

After Training Ex. 8
['Sunny' 'Warm' '?' '?' '?' '?']

After Training Ex. 9
['Sunny' 'Warm' '?' '?' '?' '?']

After Training Ex. 10
['Sunny' 'Warm' '?' '?' '?' '?']

Final Hypothesis
['Sunny' 'Warm' '?' '?' '?' '?']

Candidate Elimination Algorithm - Step-by-Step Code Description

1. Define candidate_elimination() function

A function is created with concepts and targets as inputs.

2. Initialize Specific Boundary (S)

The first training example is taken as the initial specific hypothesis.

3. Initialize General Boundary (G)

A general hypothesis containing ? for all attributes is created.

4. Display Initial Boundaries

The initial values of S and G are printed.

5. Loop through training examples

Each concept/training example is processed using a loop.

6. Check the target value

The program checks whether the current example is positive (yes) or negative.

7. Process positive example

For a positive example, each attribute is compared with the Specific Boundary.

8. Generalize Specific Boundary

If an attribute differs, its value in S is replaced with ?.

9. Process negative example

For a negative example, the program enters the else block and checks the attributes against the Specific Boundary.

10. Update General Boundary

New hypotheses are added to the General Boundary to represent possible general hypotheses.

11. Display S and G

The current Specific and General boundaries are displayed after each example.

12. Return the boundaries

The function returns the final specific and general boundaries.

13. Execute Candidate Elimination

The function is called using `candidate_elimination(X, y)`, and the results are stored in S and G.

14. **Display Final Specific Boundary**

The final Specific Boundary is printed. In your output, it is **[Sunny, Warm, Normal, Strong, Warm, Same]**.

15. **Display Final General Boundary**

The program uses a for loop to print each hypothesis in the final General Boundary.

In []:

```
def candidate_elimination(concepts, targets):
    specific = concepts[0].copy()
    general = [["?" for i in range(len(specific))]]

    print("\nInitial S:",
          specific)

    print("Initial G:",
          general)
    for i, h in
    enumerate(concepts):

        if targets[i] == "yes":
            # Positive example
            for x in
            range(len(specific)):
                if h[x] != specific[x]:
                    specific[x] = "?"
```

In

[17]:

```

else:
    # Negative example
    for x in
    range(len(specific)):
        if h[x] != specific[x]:
            general.append(
                ["?" if j != x else
                specific[x] for j in
                range(len(specific))] )

    print("\nExample",
i + 1) print("S:",
specific)
print("G:", general)
return specific,
general

```

```

S, G = candidate_elimination(X, y)

```

```

print("\nFinal Specific
Boundary") print(S)

```

```

print("\nFinal General
Boundary") for g in G:
    print(g)

```

Initial S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm'
'Same']

Initial G: [['?', '?', '?', '?', '?', '?']]

Example 1

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?']]

Example 2

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?']]

Example 3

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'No rmal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same']]

Example 4

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'No rmal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'S ame']]

Example 5

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'No rmal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'S ame']]

'?', 'Same'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?']]

Example 6

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'Normal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['Sunny', '?', '?', '?',
'?', '?'], ['?', '?', '?', 'Strong', '?', '?']]

Example 7

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'Normal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['Sunny', '?', '?', '?',
'?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['?', '?', '?', '?',
'Warm', '?']]

Example 8

S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'Normal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['Sunny', '?', '?', '?',
'?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?',

```
'?', '?', 'Strong', '?', '?'], ['?', '?', '?', '?',
'Warm', '?'], ['Sunny',
'?', '?', '?', '?', '?'], ['?', '?', 'Normal', '?',
'?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?',
'?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'Same']]
```

Example 9

```
S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'No rmal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'S ame'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['Sunny', '?', '?', '?',
'?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['?', '?', '?', '?',
'Warm', '?'], ['Sunny',
'?', '?', '?', '?', '?'], ['?', '?', 'Normal', '?',
'?', '?'], ['?', '?', '?',
'Strong', '?', '?'], ['?', '?', '?', '?', 'Warm', '?'],
['?', '?', '?', '?',
'?', 'Same'], ['?', '?', '?', '?', 'Warm', '?']]
```

Example 10

```
S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
G: [['?', '?', '?', '?', '?', '?'], ['?', '?',
'Normal', '?', '?', '?'], ['Sunn y', '?', '?', '?',
'?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', 'No rmal', '?', '?', '?'], ['?', '?', '?', '?',
'?', 'Same'], ['?', '?', 'Normal', '?', '?', '?'],
['?', '?', '?', '?', 'Warm', '?'], ['?', '?', '?', '?',
'?', 'S ame'], ['?', 'Warm', '?', '?', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['Sunny', '?', '?', '?',
'?', '?'], ['?', '?', '?', 'Strong', '?', '?'], ['?',
'?', '?', 'Strong', '?', '?'], ['?', '?', '?', '?',
'Warm', '?'], ['Sunny',
```

```
'?', '?', '?', '?', '?'], ['?', '?', 'Normal', '?',
'?', '?'], ['?', '?', '?',
'Strong', '?', '?'], ['?', '?', '?', '?', 'Warm', '?'],
['?', '?', '?', '?',
'?', 'Same'], ['?', '?', '?', '?', 'Warm', '?'], ['?',
'?', 'Normal', '?', '?',
''], ['?', '?', '?', 'Strong', '?', '']]
```

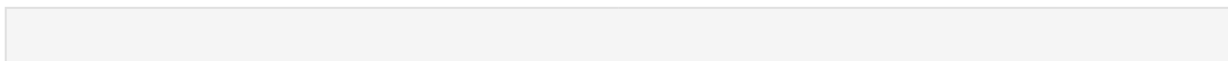
Final Specific Boundary

```
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
```

Final General Boundary

```
['?', '?', '?', '?', '?', '?']
['?', '?', 'Normal', '?', '?', '?']
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', 'Normal', '?', '?', '?']
['?', '?', '?', '?', '?', 'Same']
['?', '?', 'Normal', '?', '?', '?']
['?', '?', '?', '?', 'Warm', '?']
['?', '?', '?', '?', '?', 'Same']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', '?', 'Strong', '?', '?']
['Sunny', '?', '?', '?', '?', '?']
['?', '?', '?', 'Strong', '?', '?']
['?', '?', '?', 'Strong', '?', '?']
['?', '?', '?', '?', 'Warm',
''] ['Sunny', '?', '?', '?',
'?', '?']
['?', '?', 'Normal', '?', '?', '?']
['?', '?', '?', 'Strong', '?', '?']
['?', '?', '?', '?', 'Warm', '?']
['?', '?', '?', '?', '?', 'Same']
['?', '?', '?', '?', 'Warm', '?']
['?', '?', 'Normal', '?', '?', '?']
['?', '?', '?', 'Strong', '?', '?']
```

In []:



Viva questions

1 What is Find-S?

Find-S is a concept learning algorithm that finds the **most specific hypothesis** consistent with positive training examples.

2. Which examples does Find-S consider?

It considers only **positive (Yes) examples**.

3. Does Find-S consider negative examples?

No, negative examples are ignored.

4. How is the initial hypothesis selected?

The first positive example is used as the initial hypothesis.

5. What does ? mean in the hypothesis?

It means **any value is acceptable** for that attribute.

6. When is an attribute changed to ??

When its value differs between the current hypothesis and a positive example.

7. What is the final Find-S hypothesis in your program?

[Sunny, Warm, ?, ?, ?, ?]

8. Why does the Find-S hypothesis become more general?

Because different attribute values in positive examples are replaced with

9. What is Candidate Elimination?

It is a concept learning algorithm that maintains both **Specific (S) and General (G) boundaries**.

10. What is the Specific Boundary?

It represents the **most specific hypotheses** consistent with the training data.

11. What is the General Boundary?

It represents the **most general hypotheses** consistent with the training data.

12. What is the initial Specific Boundary in your code?

The first training example:

[Sunny, Warm, Normal, Strong, Warm, Same]

13 What is the initial General Boundary?

[?, ?, ?, ?, ?, ?]

14. What happens when a positive example is encountered?

The Specific Boundary is generalized wherever necessary.

15. What happens when a negative example is encountered?

The General Boundary is specialized to exclude the negative example.

16. Why are S and G maintained?

To represent the range of hypotheses that are consistent with the training examples.